QUOTEMAX · ENGINE INTERNALS

THE QUOTE ENGINE, TRACED FROM SOURCE

> How a customer message becomes a grounded, GST-correct single fixed-price quote — every step, every table it reads, and the hard gate that drops an unsafe quote to a $99 site visit, credited straight back to the job. Diagrams below are mapped from the running code in `lib/intake/` and `lib/estimate/`, not from the summary docs.

01 · WHAT YOU ARE LOOKING AT

TWO ENGINES, ONE MONEY PATH

QuoteMax runs every job through two stages. The Intake Engine turns a voice or SMS conversation (plus any photos) into one structured `intakes` row. The Estimation Engine turns that row into a priced quote — drafting with Claude, but forcing **every price to derive from the database** and bouncing anything it can't prove to a flat $99 inspection.

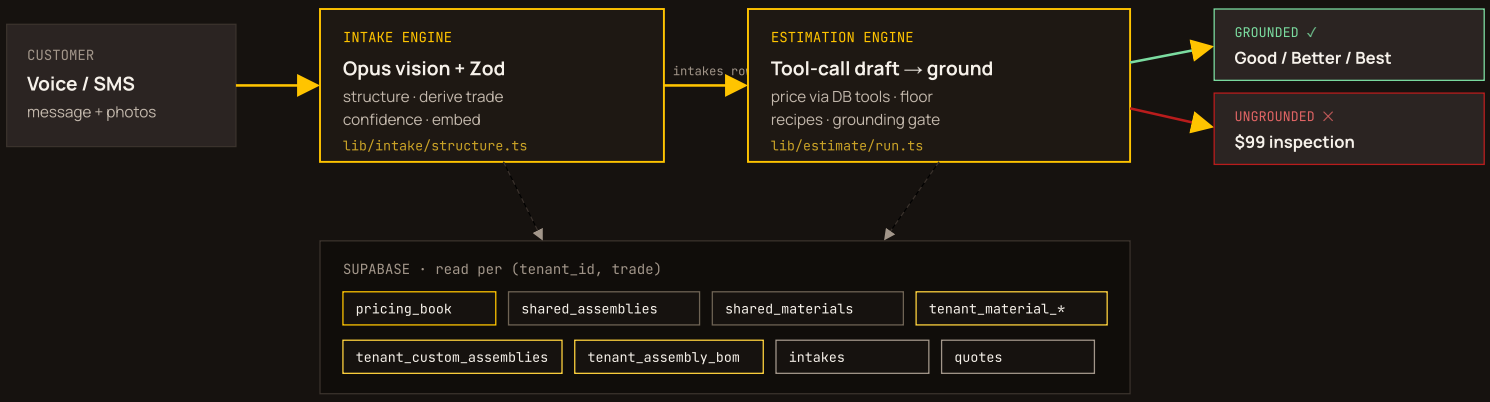
- pricing_book · the rate card
- shared_* · global catalogue
- tenant_* · operator overlay
- model call · Opus / Voyage / Gemini
- pipeline I/O · intakes / quotes / sms

> **Reading the diagrams.** Each stage card lists a READS row of coloured chips — those are the exact tables & columns that stage touches. The colour tells you *where the data lives*. Follow the orange chips to see every place a real dollar figure enters the quote.

02 · THE 10,000-FT VIEW

FROM "HI, I NEED…" TO A SENT QUOTE

Two HTTP routes do the work: `/api/intake/structure` then `/api/estimate/draft`. Both run a 3-model Claude cascade (Opus 4.8 → Opus 4.7 → Sonnet 4.6) with retry, and both read the same per-tenant, per-trade slice of the database.



▲ Both engines read the same database, always filtered to the intake's `tenant_id` + `trade`. There is no cross-tenant price fallback — an unresolved rate card routes straight to inspection.

03 · INTAKE ENGINE

CONVERSATION → ONE STRUCTURED ROW

Entry: `app/api/intake/structure/route.ts`. The model is asked for the Zod schema *minus* the `trade` field (Anthropic caps optional params at 24); trade is then derived deterministically from `job_type`. No prices are touched here — this stage only decides *what the job is* and *how confident we are*.

1

Load the conversation & photos

`route.ts · SMS: stitch sms_messages → transcript · voice: calls.transcript`

SMS path stitches every `sms_messages` turn into a transcript and de-dupes photos from both the message thread and the upload link. A 10-minute idempotency guard short-circuits if an intake already exists.

READS · `sms_conversations.conversation_state` · `sms_messages.body / photo_urls` · `calls.transcript`

2

Structure with Opus vision (no tools)

`lib/intake/structure.ts · generateObject(IntakeSchema, model cascade)`

The transcript + each photo URL (passed as `{type:'image'}` vision parts) go to Claude under a strict *grounding* system prompt: extract only what was said or seen, never invent. The prompt branches on an electrical vs plumbing hint to load the right spec-extraction rules (color temp, dimmable, weatherproof, supplied_by…).

MODEL · Opus 4.8 → 4.7 → Sonnet 4.6 · vision: photo URLs

STRICT GROUNDING · TEMPERATURE OMITTED

3

Derive trade, embed, resolve customer memory

schema.ts:deriveTradeFromJobType · embed.ts:embedIntake · customers lookup

`job_type` in the plumbing set → `plumbing`, else `electrical`. A one-line summary is embedded by Voyage `voyage-3-large` (1024-dim) for similarity search. Known caller name/suburb/email backfill any blanks Opus couldn't hear.

READS · Voyage embeddings · 1024-dim · customers.full_name / suburb / email

4

Insert the intake & gate quality

route.ts → intakes INSERT · quality.ts:evaluateIntakeQuality

One `intakes` row is written (scope/access/property/risks/caller/timing as jsonb, plus the embedding). A quality gate decides USABLE (→ hand off to estimation) vs EMPTY (→ send a focused recovery question and stop). LOW confidence needs a real name *and* a real scope to pass.

WRITES · intakes.scope / confidence / embedding / trade

USABLE → POST /API/ESTIMATE/DRAFT

> **Traced reality vs the summary docs.** CLAUDE.md calls this "Opus 4.7 vision + tool-calling." In code it uses *structured output* (`generateObject`) with **no tools**, across a 3-model fallback cascade. Tool-calling only appears later, in the estimator. The voice path also hard-pins `trade = electrical` — a plumbing phone call would be structured with the electrical prompt.

04 · ESTIMATION ENGINE — THE CORE

WHERE PRICES & MATERIALS ACTUALLY COME FROM

Entry: `app/api/estimate/draft/route.ts` → `lib/estimate/run.ts:runEstimation`. Claude drafts the quote, but it can **only** get a number by calling a database tool — and several deterministic passes mutate the draft before and after the model. Follow the chips: every place a dollar enters is an orange (rate-card), teal (shared catalogue) or peach (tenant overlay) chip.

1

Resolve the rate card — or stop

route.ts → pricing-book.ts:resolvePricingBookForIntake

Loads the single `pricing_book` row for this exact `(tenant_id, trade)`. **No cross-tenant fallback** — if there's no book, the estimator never runs and the job goes straight to $99 inspection. This row defines labour rates, markup band, call-out minimum, min-labour floor and GST status.

READS · pricing_book.hourly_rate · default_markup_pct · call_out_minimum · min_labour_hours · gst_registered

2

Build soft hints into the user message

run.ts:116-176 · rag.ts · catalogue.ts · BOM hint

Advisory context is added to the *user* message only (never the cached system prompt, never the validator): similar past quotes (RAG), preferred brands, the operator's branded catalogue + Good/Better/Best ladder, and a bill-of-materials hint. These *steer* material choice; they can't authorise a price.

READS · tenant_material_preferences.preferred_brand · tenant_material_catalogue · tenant_tier_ladder · tenant_assembly_bom · shared_assembly_bom · quotes (past, RAG)

3

Opus drafts — pricing is tool-calling only

run.ts:190-226 generateText · tools.ts:makeTools(tenantId)

The model runs with four tools and `stepCountIs(10)`. It must obtain every price through them. `lookupAssembly` and `lookupMaterial` search the DB (token + synonym expansion), then a Voyage cross-encoder **reranks** results so the best match is row #1. `applyMarkup` applies the rate-card markup; `flagInspectionNeeded` lets it bail.

TOOL READS · shared_assemblies · shared_materials · tenant_custom_assemblies (enabled) · tenant_material_catalogue (active) · Voyage rerank-2.5

ALWAYS_INSPECTION ROWS EXCLUDED · WP5: CUSTOMER-SUPPLY PRICE FLIP

4

Optional: deterministic BOM rebuild

run.ts:279-318 · deterministic-bom.ts · catalogue.ts:chooseMaterial · FLAG DETERMINISTIC_BOM

When enabled, this *replaces* Opus's line items entirely: for each tier it picks a material per BOM category (tier-ladder hit › brand/range score › shared fallback) and rebuilds labour from the assembly's hours × the rate card. Off by default — Opus's draft stands.

READS · tenant_assembly_bom.material_category · tenant_material_catalogue.unit_price_ex_gst · tenant_assembly_overrides · hourly_rate · shared_materials (fallback)

5

Minimum-labour floor

run.ts:326 · min-labour.ts:applyMinLabourFloor

Tops labour up to `min_labour_hours` at the hourly rate, *before* grounding, so a tiny-but-correct job (one downlight) is charged the tradie's minimum instead of being bounced. Never undercharges, never fabricates a part price.

READS · pricing_book.min_labour_hours · pricing_book.hourly_rate

6

Price-bands recipe merge

run.ts:344-432 · merge-recipes.ts · price-bands.ts (mig 074)

If the chosen assembly carries a `price_recipe`, the customer's slot answers (e.g. "12 m cable run", "double-storey") are evaluated to *append* extra labour / cable / risk flags or even *swap* the base assembly. Every band must yield a price — there is no "route to inspection" band here.

READS · shared_assemblies.price_recipe · tenant_custom_assemblies.price_recipe · conversation_state.slots · hourly_rate / markup

7

Grounding validation — the hard backstop

run.ts:734 loadCandidatePrices · validate.ts:validateQuoteGrounding

Loads every legal price for this `(tenant, trade)` and checks **each line**: labour must equal a rate-card rate; materials/assemblies must match a catalogue row's raw or marked-up price (±$0.50) *and* share its product category. A typed `source:'material:<uuid>'` takes a strict by-id path. **Any single failure nulls all three tiers** → $99 inspection.

CANDIDATE SET · shared_materials · shared_assemblies · tenant_custom_assemblies · tenant_material_catalogue · pricing_book rates + markup band

PASS → AUTO-QUOTE | FAIL → DOWNGRADE WHOLE QUOTE

8

Persist, route & send

route.ts:246-820 · routing/decide.ts · early-bird · stripe · dispatch

The validated draft is written to `quotes` (line items denormalised into `good/better/best` jsonb). GST is added if the book is GST-registered; an early-bird offer is stamped from `overlays.early_bird`; Stripe deposit links (30% per tier, or one $99) are created; then the customer SMS sends — unless the review policy holds it for the tradie.

WRITES · quotes.good / better / best · quotes.total_inc_gst / routing_decision · gst_registered · overlays.early_bird · review_policy

> **The draft is rarely "pure Opus."** Stages 4–6 are deterministic passes that mutate the line items around the model. In the default config (BOM & product-options flags off) Opus tool-calling really is the live price source — but the min-labour floor and recipe merge always run, and the grounding validator is the only thing standing between a hallucinated price and the customer.

05 · THE PROVENANCE TABLE

EVERY NUMBER ON A QUOTE, AND ITS SOURCE

This is the heart of "accurate & working properly": each element of a line item maps to one authoritative table column. If a value can't be traced to this list, the grounding gate rejects it.

QUOTE ELEMENT	SOURCE · TABLE.COLUMN	HOW IT'S RESOLVED
Labour rate	pricing_book.hourly_rate	Standard hour. Apprentice/senior use `apprentice_rate/senior_rate`.
After-hours labour	hourly_rate × after_hours_multiplier	Only when the line is *source-tagged* emergency/after-hours (description text can't trigger it).
Call-out minimum	pricing_book.call_out_minimum	Accepted for `source:'callout'` lines.

QUOTE ELEMENT	SOURCE · TABLE.COLUMN	HOW IT'S RESOLVED
Markup %	● pricing_book.default_markup_pct	Validator accepts raw, or ±5pp around this value, on material prices.
Min-labour floor	● pricing_book.min_labour_hours	Small jobs topped up to this many hours at the hourly rate.
Material price (raw)	● shared_materials.default_unit_price_ex_gst ● tenant_material_catalogue.unit_price_ex_gst	The catalogue cost; quote shows it raw or × markup.
Customer-supply price	● tenant_material_catalogue.customer_supply_price_ex_gst	Install-only price when the customer provides the fitting (WP5). No price → row dropped (no double-billing).
Assembly base price	● shared_assemblies.default_unit_price_ex_gst ● tenant_custom_assemblies	Bundled job price; `always_inspection=true` rows are excluded.
Which material per tier	● tenant_tier_ladder + ● tenant_material_catalogue	Good/Better/Best ladder hit wins; else brand/range/preferred score; else shared fallback (`chooseMaterial`).
What an assembly needs	● tenant_assembly_bom / ● shared_assembly_bom	Bill-of-materials categories & quantities per assembly.
Slot-driven extras	● shared_assemblies.price_recipe ● tenant_custom_assemblies.price_recipe	Extra labour/cable/risk from the customer's answers (mig 074).
Brand steer (soft)	● tenant_material_preferences.preferred_brand	Hint only — never a hard filter, never grounds a price.
GST (10%)	● pricing_book.gst_registered	Added to the selected tier subtotal when true.
Early-bird discount	● pricing_book.overlays.early_bird	Clamped to a 15% ceiling; realised at the /book route.
Inspection fallback	● INSPECTION_TOTAL_INC_GST = $99	Hardcoded; forced whenever grounding fails or inspection is flagged.

06 · THE SAFETY GATE

HOW A PRICE IS PROVEN — OR REJECTED

Grounding is deterministic and makes no LLM call. It is the reason a hallucinated or mis-categorised price can never reach a customer: the worst case is a safe $99 inspection, not a wrong number.

DRAFT LINE
`desc · qty · unit · unit_price · source`

→

TYPED UUID?
`source = 'material:<id>'?`

→

MATCH CANDIDATE
`price = raw or ×markup ±$0.50`

STRICT PATH — TYPED ROW ID

The line must point at a real row in the tenant+trade candidate set, and its price must equal one of that exact row's variants (raw / −5pp / default / +5pp markup) within $0.50. Wrong trade, deleted, or fabricated id → `FAIL` .

LOOSE PATH — PRICE + CATEGORY

Price must match some candidate within $0.50 **and** the line's category (from the description) must overlap that row's category. Stops a "smoke alarm" line being priced from a "downlight" row at the same dollar figure.

`ALL LINES PASS`

Auto-quote

Good/Better/Best survive. Render-only enrichment links catalogue photos. GST + early-bird + Stripe links applied. Customer gets a priced quote.

`≥1 LINE FAILS · OR DUPLICATE ANCHOR · OR BELOW LABOUR FLOOR`

Whole quote → $99 inspection

All three tiers nulled (defence-in-depth, even if Opus emitted numbers), `needs_inspection=true`, each failure stamped into `risk_flags`, one $99 Stripe session.

> **Also enforced:** a per-tier labour-hours floor (re-checked inside the validator) and a duplicate-line guard (D-1) that catches the same product billed twice — once at raw cost and once marked-up — a real incident (Dux Proflo 315L billed at both $1,645 and $2,237.20) that the per-line checks each accepted individually.

07 · THE DATABASE, GROUPED BY ROLE

WHAT LIVES WHERE

Every pricing read is scoped by `trade` and `tenant_id`. Key columns are outlined. Types are from `sql/init.sql` + migrations (e.g. `embedding` is now `vector(1024)` after mig 057, though init.sql still reads 1536 — see traps).

THE RATE CARD

`pricing_book`

per (tenant_id, trade) · numeric

`tenant_id` `trade` `hourly_rate` `apprentice_rate` `senior_rate`

call_out_minimum | default_markup_pct
risk_buffer_pct | min_labour_hours
after_hours_multiplier
gst_registered | overlays
review_policy | quote_display

SHARED CATALOGUE (GLOBAL)

shared_assemblies
job-level bundles

id | trade | name
default_unit_price_ex_gst
default_labour_hours | category
properties | always_inspection
price_recipe | clarifying_questions

shared_materials
products

id | trade | name | brand | unit
default_unit_price_ex_gst | category
properties

shared_assembly_bom
baseline recipe

assembly_id | material_category
quantity | required | sort

TENANT OVERLAY (OPERATOR-OWNED)

tenant_material_catalogue
mig 028 · real branded stock

id | tenant_id | trade | category
name | brand | range_series
unit_price_ex_gst
customer_supply_price_ex_gst
tier_hint | active | image_path

tenant_custom_assemblies
mig 023

id | tenant_id | trade | name
default_unit_price_ex_gst
default_labour_hours | enabled
always_inspection | price_recipe

tenant_assembly_bom
mig 031 · own recipe

tenant_id | assembly_id
material_category | quantity
required

tenant_tier_ladder
mig 043 · G/B/B map

tenant_id | category | tier
catalogue_id

tenant_assembly_overrides
per-assembly tweaks

tenant_id | assembly_id
labour_hours_override
markup_pct_override

tenant_material_preferences
mig 022 · soft hint

tenant_id | category
preferred_brand

tenant_service_offerings
services on/off

tenant_id | assembly_id | enabled

PIPELINE I/O

intakes
structured job

id | tenant_id | trade | job_type
scope (jsonb) | risks | confidence
inspection_required
embedding vector(1024) | photo_paths

quotes
output · G/B/B denormalised

id | intake_id | tenant_id
good / better / best (jsonb)
selected_tier | subtotal_ex_gst | gst
total_inc_gst | needs_inspection
routing_decision | share_token
stripe_links

08 · CUSTOMER X MEETS QUOTEMAX

ONE JOB, END TO END

An electrical tenant ("Sparky", rate card: `$110/hr · 28% markup · min 2.0 h · GST-registered`). The customer texts about downlights. Watch it flow through both engines into one clear fixed price.

● SMS · +61 4XX XXX XXX

Hi, can I get a price to swap 6 old halogen downlights for LED in my living room?

Happy to help! Are you after warm white or cool white, and do you want them dimmable?

Warm white, and yes dimmable please. Existing wiring's all there.

Great. What's the address & your name so I can sort the quote?

Jordan — 12 Banksia St, Chatswood.

Thanks Jordan 👍 Sending your quote through now.

① INTAKE ENGINE OUTPUT → INTAKES ROW

```
// derived, not asked: trade ← job_type 'downlights' { "trade":
"electrical", "job_type": "downlights", "suburb": "Chatswood",
"address": "12 Banksia St", "scope": { "item_count": 6,
"is_new_install": false, "existing_wiring": true, "indoor_outdoor":
"indoor", "specs": { "color_temp": "warm_white", "dimmable": true }
}, "inspection_required": false, "caller": { "name": "Jordan" },
"confidence": "HIGH" }
```

② ESTIMATION ENGINE TRACE

`lookupAssembly("replace downlight")` → reranked #1 = `shared_assemblies · "Replace downlight (LED)"` · 0.5 h each

`lookupMaterial("LED downlight warm white dimmable")` → 3 tiers via `tenant_tier_ladder` + `tenant_material_catalogue`

`applyMarkup(raw, 28)` on each material · labour = 6 × 0.5 h = **3.0 h** × $110

`MIN-LABOUR 3.0 ≥ 2.0 ✓` `GROUNDING ✓ ALL LINES`

`SELECTED_TIER = BETTER`

③ THE QUOTE JORDAN RECEIVES `/Q/<SHARE_TOKEN>` · PRICE SHOWN INC-GST

YOUR FIXED PRICE ✓

LED DOWNLIGHT SWAP

$599.54 inc GST

6× 10W LED downlight, warm white, dimmable `6 each · $35.84 extenant_material_catalogue · raw $28.00 ×1.28`	$215.04
Electrician labour `3.0 hr · $110.00 expricing_book.hourly_rate`	$330.00
Subtotal (ex GST)	$545.04
GST 10%	$54.50
Total inc GST	**$599.54**

30% deposit to book	$179.86

One clear fixed price, no haggling. Internally the engine still resolves the full `good / better / best` jsonb via the tier ladder (`chooseMaterial` / `tenant_tier_ladder`); the operator's display mode then publishes the selected tier as a single fixed price. Every line above is grounded: material prices are a catalogue row's raw cost × the rate-card markup; labour equals the rate-card hourly; the total = subtotal + 10% GST.

> **The alternate ending.** Had Jordan said "I want to upgrade my switchboard too," the intake sets `inspection_required=true` (or the assembly is `always_inspection`) — Opus calls `flagInspectionNeeded`, validation is skipped, all tiers are nulled, and Jordan instead gets one link: a **$99 site inspection**. The system never guesses a price it can't prove.

09 · MAKE IT ACCURATE & WORKING PROPERLY

WHAT THE TRACE SURFACED

These are real findings from reading the running code (each adversarially re-verified). None breaks the safety guarantee — but each is a place where an *accurate* price could be silently bounced to inspection, or a wrong one slip a check. Ordered by impact on estimation accuracy.

1 **The markup tool defaults to 28% — wrong for plumbing**

`applyMarkup` falls back to a hardcoded 28% (electrical median) if Opus omits `markupPct`. On a plumbing book (~15–20%) a 28% line is 8–13pp outside the validator's ±5pp band, so it **fails grounding and bounces a correct job to $99**. It fails safe, but it's a silent conversion-killer.

```
fix → make markupPct required, or inject pricing_book.default_markup_pct into the tool closure
```

2 **The validator checks unit price, never quantity or subtotal**

Grounding proves each `unit_price_ex_gst`, but nothing reconciles `quantity × unit` or checks that a tier's `subtotal_ex_gst` equals the sum of its lines. A correct unit price with an absurd quantity, or a drifted subtotal, passes.

```
fix → assert subtotal == Σ(line totals) and sanity-bound quantity per job_type
```

3 **Embedding dimension drift mutes RAG**

Live vectors are 1024-dim (mig 057) but `init.sql`, `rag.ts` headers and the intake log still say 1536. More importantly, mig 057 NULLed all embeddings and the re-embed backfill stalled (≈3/70) — so most past intakes contribute **zero** similar-quote context, silently, not as an error.

```
fix → finish the Voyage backfill; correct the stale 1536 labels
```

4 **Strict-UUID path skips the category check**

A line tagged `source:'material:<id>'` is accepted on price alone — the semantic category guard only runs on the loose path. A real row id with a mismatched description would price through unchecked.

```
fix → keep the category overlap assertion on both paths
```

5 **Min-labour floor is case-sensitive; validator isn't**

`min-labour.ts` sums only `unit==='hr'` (exact case) but the validator normalises case. A tier with `'HR'` lines is summed by the validator yet not topped up by the floor — a misconfigured hourly rate then yields a confusing inspection downgrade instead of a clear error.

`fix → normalise unit case in both; surface rate-card misconfig explicitly`

---

6 **Voice calls are hard-pinned to electrical**

The voice path never derives a trade hint for the structuring prompt — a plumbing phone call is structured with electrical spec rules. Low impact today (Vapi pilot is electrical-only) but a correctness landmine for multi-trade voice.

`fix → derive tradeHint from the call's job_type before structuring`

---

7 **Gas-HWS regulatory inspection is flag-gated off at intake**

The AS/NZS 5601 "force inspection for gas hot water" override at intake is behind `FORCE_GAS_HWS_SITE_VISIT` (default off); safety routing relies on the downstream catalogue's `always_inspection` (mig 068) instead. Two backstops that should agree explicitly.

`fix → make the regulatory rule data-driven and always-on, not env-gated`

Traced from source on 2026-06-01 across `lib/intake/{structure,schema,embed,quality}.ts`, `lib/estimate/{run,tools,validate,pricing-book,catalogue,merge-recipes,min-labour,rag,rerank}.ts`, `lib/routing/decide.ts` and `app/api/{intake/structure,estimate/draft}/route.ts`, cross-checked against `sql/init.sql` + migrations 022–074. Six subsystems mapped and adversarially re-verified; all reported accurate.